

Métodos de arrays, transformación de información e introducción al testing de lógica en JavaScript

Enfoque de la sesión

En esta sesión vas a trabajar una de las partes más importantes del JavaScript moderno: **transformar información de manera clara, expresiva y mantenible**. El objetivo no es solo conocer métodos de arrays, sino aprender a utilizarlos para recorrer, filtrar, buscar, comprobar, ordenar y reformular datos sin caer en soluciones rígidas o difíciles de leer. Además, esta sesión introduce dos ideas fundamentales para todo el bloque: **inmutabilidad y funciones puras**, junto con un primer contacto con el **testing de lógica**.

1. Introducción

En la sesión anterior trabajaste con arrays, objetos, modelado de datos, desestructuración, `spread`, `optional chaining`, `nullish coalescing` y JSON. Eso te permitió representar la información de forma estructurada. Sin embargo, representar datos es solo el principio. Una aplicación real necesita además **operar con ellos**.

En el trabajo diario con JavaScript aparecen constantemente tareas como estas:

- recorrer una colección;
- extraer solo algunos elementos;
- transformar cada valor;
- localizar un registro concreto;
- comprobar si se cumple una condición;
- agrupar o resumir información;
- ordenar resultados;
- eliminar duplicados;
- generar una nueva colección a partir de otra.

JavaScript ofrece para ello un conjunto de métodos de arrays que, en muchos escenarios, expresan mejor la intención que un bucle manual. Esto no significa que los bucles dejen de ser válidos, sino que el lenguaje proporciona herramientas que ayudan a escribir código más legible y más cercano al problema que quieres resolver.

Esta sesión es especialmente importante porque aquí empieza a consolidarse una forma de trabajar que será central más adelante en DOM, en asincronía y especialmente en React:

- partir de una colección de datos;
 - transformarla con criterios claros;
 - obtener otra versión más útil para la interfaz o la lógica.
-

2. Recorrer una colección: `forEach()`

2.1. Qué hace `forEach()`

`forEach()` ejecuta una función una vez por cada elemento del array.

```
const nombres = ["Ana", "Luis", "María"];

nombres.forEach((nombre) => {
  console.log(nombre);
});
```

2.2. Cuándo es útil

Conviene utilizar `forEach()` cuando:

- necesitas recorrer un array completo;
- quieres ejecutar una acción con cada elemento;
- no necesitas generar un nuevo array como resultado.

2.3. Qué no hace

`forEach()` no devuelve un nuevo array transformado. Su propósito es recorrer y ejecutar lógica asociada a cada elemento.

```
const numeros = [1, 2, 3];

const resultado = numeros.forEach((numero) => numero * 2);

console.log(resultado); // undefined
```

2.4. Parámetros habituales

La función callback puede recibir:

- el elemento;
- el índice;

- el array completo.

```
const nombres = ["Ana", "Luis", "María"];

nombres.forEach((nombre, indice) => {
  console.log(`${indice}: ${nombre}`);
});
```

2.5. Cuándo no conviene usarlo

Si tu objetivo es construir una nueva colección transformada, `forEach()` no suele ser la mejor opción. En ese caso conviene pensar si lo que realmente necesitas es `map()`.

2.6. Comparación con `for...of`

Aunque ambas opciones permiten recorrer colecciones, no ofrecen el mismo nivel de control ni se comportan igual en todos los contextos. La comparación siguiente te ayudará a ver en qué situaciones resulta más conveniente cada una.

Característica	<code>for...of</code>	<code>forEach</code>
Tipo	Bucle	Método de array
<code>break / continue</code>	✅ Sí	❌ No
<code>async/await</code>	✅ Funciona	❌ No espera
Control del flujo	Alto	Limitado
Legibilidad	Muy clara	Más declarativa

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.forEach()`

3. Transformar elementos: `map()`

3.1. Qué hace `map()`

`map()` crea un nuevo array a partir del original, transformando cada elemento mediante una función.

```
const numeros = [1, 2, 3, 4];

const duplicados = numeros.map((numero) => numero * 2);

console.log(duplicados); // [2, 4, 6, 8]
```

3.2. Cuándo usarlo

Conviene usar `map()` cuando:

- cada elemento del array original debe convertirse en otro valor;
- quieres producir un nuevo array;
- la transformación afecta a todos los elementos.

3.3. Ejemplo con objetos

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 }
];

const nombres = productos.map((producto) => producto.nombre);

console.log(nombres); // ["Teclado", "Ratón"]
```

3.4. Transformar objetos

También puedes crear nuevas versiones de objetos dentro del array.

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 }
];

const productosConIVA = productos.map((producto) => {
  return {
    ...producto,
    precioConIVA: producto.precio * 1.21
  };
});

console.log(productosConIVA);
```

3.5. `map()` no modifica el array original

```
const numeros = [1, 2, 3];
const cuadrados = numeros.map((numero) => numero ** 2);

console.log(numeros); // [1, 2, 3]
console.log(cuadrados); // [1, 4, 9]
```

Idea clave

`map()` expresa muy bien la idea de **transformar una colección en otra colección**.

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.map()`

4. Filtrar elementos: `filter()`

4.1. Qué hace `filter()`

`filter()` crea un nuevo array con los elementos que cumplen una condición.

```
const numeros = [1, 2, 3, 4, 5, 6];

const pares = numeros.filter((numero) => numero % 2 === 0);

console.log(pares); // [2, 4, 6]
```

4.2. Cuándo usarlo

Conviene usar `filter()` cuando:

- solo quieres conservar parte del array;
- necesitas descartar elementos según una condición;
- quieres obtener un subconjunto de datos.

4.3. Ejemplo con objetos

```
const tareas = [
  { titulo: "Estudiar", completada: true },
```

```
{ titulo: "Practicar", completada: false },  
{ titulo: "Repasar", completada: true }  
];  
  
const completadas = tareas.filter((tarea) => tarea.completada);  
  
console.log(completadas);
```

4.4. filter() siempre devuelve un array

Aunque solo haya un resultado o ninguno, el resultado sigue siendo un array.

```
const numeros = [1, 2, 3];  
const mayoresQueDiez = numeros.filter((numero) => numero > 10);  
  
console.log(mayoresQueDiez); // []
```

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.filter()`
-

5. Buscar un único elemento: find()

5.1. Qué hace find()

`find()` devuelve el primer elemento que cumple una condición.

```
const productos = [  
  { id: 1, nombre: "Teclado" },  
  { id: 2, nombre: "Ratón" },  
  { id: 3, nombre: "Monitor" }  
];  
  
const producto = productos.find((item) => item.id === 2);  
  
console.log(producto);
```

5.2. Diferencia respecto a filter()

- `find()` devuelve **un elemento** o `undefined`;
- `filter()` devuelve **un array**.

5.3. Cuando no encuentra coincidencia

```
const productos = [
  { id: 1, nombre: "Teclado" },
  { id: 2, nombre: "Ratón" }
];

const producto = productos.find((item) => item.id === 10);

console.log(producto); // undefined
```

5.4. Cuándo resulta más apropiado

Usa `find()` cuando conceptualmente estás buscando **un único resultado**. Si utilizas `filter()` para ese caso, el código puede funcionar, pero expresa peor la intención.

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.find()`

6. Comprobar condiciones: `some()` y `every()`

6.1. `some()`

`some()` comprueba si **al menos un elemento** cumple una condición.

```
const notas = [4, 7, 6, 8];

const haySuspendos = notas.some((nota) => nota < 5);

console.log(haySuspendos); // true
```

6.2. `every()`

`every()` comprueba si **todos los elementos** cumplen una condición.

```
const notas = [6, 7, 8, 9];

const todasAprobadas = notas.every((nota) => nota >= 5);

console.log(todasAprobadas); // true
```

6.3. Cuándo son útiles

- `some()` cuando quieres saber si existe algún caso;
- `every()` cuando necesitas validar el conjunto completo.

6.4. Ejemplo con objetos

```
const productos = [  
  { nombre: "Teclado", disponible: true },  
  { nombre: "Ratón", disponible: true },  
  { nombre: "Monitor", disponible: false }  
];  
  
const hayAgotados = productos.some((producto) => !producto.disponible);  
const todosDisponibles = productos.every((producto) => producto.disponible);  
  
console.log(hayAgotados);  
console.log(todosDisponibles);
```

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.some()`
 - MDN Web Docs — `Array.prototype.every()`
-

7. Reducir una colección a un valor: `reduce()`

7.1. Qué hace `reduce()`

`reduce()` recorre el array y acumula un resultado único.

```
const numeros = [1, 2, 3, 4];  
  
const sumaTotal = numeros.reduce((acumulador, numero) => {  
  return acumulador + numero;  
}, 0);  
  
console.log(sumaTotal); // 10
```

7.2. Estructura básica

La función callback de `reduce()` suele trabajar con:

- acumulador ;
- elemento actual.

El segundo argumento de `reduce()` es el valor inicial del acumulador.

7.3. Ejemplo con objetos

```
const productos = [
  { nombre: "Teclado", precio: 30 },
  { nombre: "Ratón", precio: 15 },
  { nombre: "Monitor", precio: 200 }
];

const total = productos.reduce((acumulador, producto) => {
  return acumulador + producto.precio;
}, 0);

console.log(total); // 245
```

7.4. Otro uso: contar elementos por criterio

```
const notas = [5, 8, 4, 9, 3, 7];

const aprobadas = notas.reduce((contador, nota) => {
  return nota >= 5 ? contador + 1 : contador;
}, 0);

console.log(aprobadas); // 4
```

7.5. Cuándo conviene usarlo

`reduce()` es útil cuando quieres:

- sumar valores;
- contar apariciones;
- construir una nueva estructura;
- obtener un único resultado agregado.

7.6. Precaución de legibilidad

Aunque `reduce()` es potente, no siempre es la opción más clara. Si el código resulta difícil de entender, a veces es preferible una combinación distinta de métodos o incluso un bucle más explícito.

⚠ Criterio importante

`reduce()` no es mejor por ser más “sofisticado”.

Debes usarlo cuando realmente mejora la claridad del problema.

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.reduce()`

8. Comprobar pertenencia y ordenar: `includes()` y `sort()`

8.1. `includes()`

`includes()` comprueba si un array contiene un determinado valor.

```
const colores = ["rojo", "verde", "azul"];

console.log(colores.includes("verde")); // true
console.log(colores.includes("negro")); // false
```

8.2. `sort()`

`sort()` ordena los elementos de un array. Sin embargo, conviene usarlo con cuidado.

```
const palabras = ["pera", "manzana", "uva"];
palabras.sort();

console.log(palabras);
```

8.3. Orden numérico correcto

En números, debes proporcionar una función comparadora.

```
const numeros = [10, 2, 30, 4];

numeros.sort((a, b) => a - b);

console.log(numeros); // [2, 4, 10, 30]
```

⚠️ Precaución importante sobre `sort()`

Por defecto, `sort()` **convierte los elementos a strings** y los ordena de forma lexicográfica (como texto), no numéricamente.

```
[10, 2, 5].sort();  
// Resultado: [10, 2, 5] ❌
```

Esto ocurre porque `"10"` se considera menor que `"2"` al compararse como texto.

8.4. Orden descendente

```
const numeros = [10, 2, 30, 4];  
  
numeros.sort((a, b) => b - a);  
  
console.log(numeros); // [30, 10, 4, 2]
```

8.5. Precaución importante

`sort()` modifica el array original.

```
const numeros = [10, 2, 30, 4];  
const copiaOrdenada = [...numeros].sort((a, b) => a - b);  
  
console.log(numeros); // [10, 2, 30, 4]  
console.log(copiaOrdenada); // [2, 4, 10, 30]
```

8.6. Ordenar objetos

```
const productos = [  
  { nombre: "Monitor", precio: 200 },  
  { nombre: "Ratón", precio: 15 },  
  { nombre: "Teclado", precio: 30 }  
];  
  
const ordenadosPorPrecio = [...productos].sort((a, b) => a.precio -  
b.precio);  
  
console.log(ordenadosPorPrecio);
```

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.includes()`
 - MDN Web Docs — `Array.prototype.sort()`
-

9. Eliminar duplicados: `Set`

9.1. Qué es `Set`

`Set` es una estructura que almacena valores únicos.

```
const valores = new Set([1, 2, 2, 3, 3, 3]);  
  
console.log(valores); // Set(3) {1, 2, 3}
```

9.2. Convertir un array con duplicados en uno sin duplicados

```
const numeros = [1, 2, 2, 3, 4, 4, 5];  
const sinDuplicados = [...new Set(numeros)];  
  
console.log(sinDuplicados); // [1, 2, 3, 4, 5]
```

9.3. Ejemplo con textos

```
const tiposPokemon = ["fuego", "agua", "agua", "planta", "fuego"];  
const tiposUnicos = [...new Set(tiposPokemon)];  
  
console.log(tiposUnicos);
```

9.4. Cuándo es útil

`Set` resulta útil cuando necesitas:

- eliminar duplicados;
- comprobar unicidad;
- representar colecciones donde no deben repetirse elementos.

9.5. Métodos de `Set`

Además de servir para eliminar duplicados o representar colecciones de valores únicos, `Set` dispone de varios métodos y propiedades que conviene conocer, aunque en esta fase no necesites usarlos todos con profundidad.

Método / Propiedad	Descripción	Ejemplo
<code>add(valor)</code>	Añade un elemento al <code>Set</code>	<code>set.add(5)</code>
<code>delete(valor)</code>	Elimina un elemento específico	<code>set.delete(5)</code>
<code>has(valor)</code>	Comprueba si existe un valor	<code>set.has(5)</code>
<code>clear()</code>	Elimina todos los elementos	<code>set.clear()</code>
<code>size</code>	Número de elementos del <code>Set</code>	<code>set.size</code>
<code>forEach(callback)</code>	Itera sobre los valores	<code>set.forEach(v => console.log(v))</code>
<code>values()</code>	Devuelve un iterador de valores	<code>set.values()</code>
<code>keys()</code>	Igual que <code>values()</code>	<code>set.keys()</code>
<code>entries()</code>	Devuelve pares <code>[valor, valor]</code>	<code>set.entries()</code>

Referencias específicas del subtema

- [MDN Web Docs — Set](#)

10. Asociar claves y valores: `Map`

10.1. Qué es `Map`

`Map` es una estructura de datos pensada para almacenar pares clave-valor.

```
const usuarios = new Map();

usuarios.set("u1", "Laura");
```

```
usuarios.set("u2", "Carlos");

console.log(usuarios.get("u1")); // "Laura"
```

10.2. Cuándo puede resultar útil

Esta estructura puede ser útil cuando:

- necesitas asociar claves con valores;
- quieres búsquedas directas por clave;
- la estructura del problema se parece a una tabla de correspondencias.

10.3. Ejemplo sencillo

```
const stockPorProducto = new Map([
  ["teclado", 12],
  ["raton", 20],
  ["monitor", 5]
]);

console.log(stockPorProducto.get("monitor")); // 5
```

10.4. Criterio didáctico

En esta fase no necesitas usar `Map` en todos tus ejercicios, pero sí conviene conocerlo y entender qué problema resuelve, porque no todos los escenarios se modelan igual de bien con arrays o con objetos literales.

10.5. Comparación con objetos

Para entender mejor qué aporta `Map`, conviene compararlo con el objeto literal tradicional. Ambas estructuras permiten asociar información, pero no resuelven exactamente los mismos escenarios de la misma manera.

Característica	Objeto <code>{}</code>	Map
Tipo de claves	String / Symbol	Cualquier tipo
Orden de elementos	No garantizado (históricamente)*	Sí, orden de inserción
Iteración	Necesita <code>Object.keys()</code> etc.	Iterable directamente (<code>for...of</code>)

Característica	Objeto {}	Map
Rendimiento	Bueno para pocos datos	Mejor para muchos elementos
Métodos integrados	Limitados	Muy completos (set , get , etc.)
Tamaño (size)	Manual	.size directo

10.6. Métodos de Map

Como ocurre con otras estructuras del lenguaje, Map dispone de un pequeño conjunto de métodos y propiedades propios. En esta tabla tienes una referencia rápida de los más importantes.

Método / Propiedad	Descripción	Ejemplo
set(clave, valor)	Añade o actualiza un par clave-valor	map.set('nombre', 'Ana')
get(clave)	Devuelve el valor asociado a la clave	map.get('nombre') // 'Ana'
has(clave)	Comprueba si existe la clave	map.has('nombre') // true
delete(clave)	Elimina un par clave-valor	map.delete('nombre')
clear()	Elimina todos los elementos	map.clear()
size	Número de elementos en el Map	map.size
forEach(callback)	Itera sobre los pares [clave, valor]	map.forEach((v,k) => console.log(k,v))
keys()	Devuelve un iterador con las claves	map.keys()

Método / Propiedad	Descripción	Ejemplo
<code>values()</code>	Devuelve un iterador con los valores	<code>map.values()</code>
<code>entries()</code>	Devuelve un iterador con pares [clave, valor]	<code>map.entries()</code>

Referencias específicas del subtema

- MDN Web Docs — `Map`

11. Inmutabilidad: evitar modificaciones accidentales

11.1. Qué significa inmutabilidad

Trabajar de forma inmutable significa evitar cambiar directamente una estructura existente cuando lo que necesitas es producir una nueva versión.

11.2. Ejemplo de mutación directa

```
const numeros = [1, 2, 3];
numeros.push(4);

console.log(numeros); // [1, 2, 3, 4]
```

Aquí el array original ha sido modificado.

11.3. Alternativa inmutable

```
const numeros = [1, 2, 3];
const nuevosNumeros = [...numeros, 4];

console.log(numeros); // [1, 2, 3]
console.log(nuevosNumeros); // [1, 2, 3, 4]
```

11.4. Ejemplo con objetos


```
const producto = {
  nombre: "Teclado",
  precio: 30
};

const productoActualizado = {
  ...producto,
  precio: 25
};

console.log(producto);           // { nombre: "Teclado", precio: 30 }
console.log(productoActualizado); // { nombre: "Teclado", precio: 25 }
```

11.5. Por qué importa

La inmutabilidad ayuda a:

- evitar efectos secundarios inesperados;
- mantener la lógica más clara;
- facilitar pruebas;
- trabajar mejor con interfaces reactivas.

Idea clave

En muchos casos, en lugar de pensar “voy a cambiar este dato”, conviene pensar “voy a construir una nueva versión de este dato”.

Referencias específicas del subtema

- MDN Web Docs — Spread syntax
- MDN Web Docs — `Array.prototype.concat()`

12. Funciones puras

12.1. Qué es una función pura

Una función pura:

- devuelve siempre el mismo resultado para la misma entrada;
- no modifica datos externos;

- no depende de elementos imprevisibles del entorno.

12.2. Ejemplo de función pura

```
function calcularDescuento(precio, descuento) {  
  return precio - descuento;  
}
```

Si llamas a esa función con los mismos argumentos, el resultado será siempre el mismo.

12.3. Ejemplo de función no pura

```
let descuentoGlobal = 10;  
  
function calcularPrecio(precio) {  
  return precio - descuentoGlobal;  
}
```

Aquí el resultado depende de una variable externa que podría cambiar.

12.4. Otro ejemplo de no pureza por mutación

```
function agregarEtiqueta(productos) {  
  productos.push("nuevo");  
  return productos;  
}
```

Esta función modifica el array recibido.

12.5. Versión más segura

```
function agregarEtiqueta(productos) {  
  return [ ...productos, "nuevo"];  
}
```

12.6. Por qué importan las funciones puras

Las funciones puras son más fáciles de:

- entender;
- reutilizar;
- probar;

- mantener.

Referencias específicas del subtema

- MDN Web Docs — Functions
 - MDN Web Docs — Side effect
-

13. Primer contacto con testing de lógica

13.1. Qué significa comprobar una función

Antes de entrar en herramientas de testing, conviene entender algo más básico: una función debería poder comprobarse a partir de sus entradas y sus salidas.

13.2. Ejemplo simple

```
function duplicar(numero) {  
  return numero * 2;  
}  
  
console.log(duplicar(4)); // debería devolver 8  
console.log(duplicar(7)); // debería devolver 14
```

13.3. Qué estás haciendo aquí realmente

Estás verificando que:

- dada una entrada concreta;
- la función produzca la salida esperada.

13.4. Ejemplo con `console.assert()`

```
function duplicar(numero) {  
  return numero * 2;  
}  
  
console.assert(duplicar(4) === 8, "Error: duplicar(4) debería devolver 8");  
console.assert(duplicar(7) === 14, "Error: duplicar(7) debería devolver 14");
```

13.5. Ejemplo con función de filtrado

```
function obtenerCompletadas(tareas) {  
  return tareas.filter((tarea) => tarea.completada);  
}  
  
const tareas = [  
  { titulo: "Estudiar", completada: true },  
  { titulo: "Practicar", completada: false }  
];  
  
console.log(obtenerCompletadas(tareas));
```

13.6. Qué debes acostumbrarte a pensar

Cuando escribas una función de transformación de datos, pregúntate:

- ¿qué recibe exactamente?
- ¿qué debería devolver?
- ¿devuelve siempre lo mismo para la misma entrada?
- ¿está modificando algo que no debería?

Alcance en esta sesión

En esta sesión no se trata todavía de desarrollar un bloque formal de testing con profundidad, sino de acostumbrarte a escribir lógica **comprobable** y bien separada.

Referencias específicas del subtema

- MDN Web Docs — Functions
- MDN Web Docs — `console.assert()`

14. Encadenamiento de métodos

14.1. Qué significa encadenar

Puedes usar varios métodos seguidos cuando el resultado de uno sirve como entrada del siguiente.

```
const numeros = [1, 2, 3, 4, 5, 6];  
  
const resultado = numeros
```

```
.filter((numero) => numero % 2 === 0)
.map((numero) => numero * 10);

console.log(resultado); // [20, 40, 60]
```

14.2. Ventaja

El encadenamiento puede hacer el código más expresivo porque muestra una secuencia clara de transformación.

14.3. Precaución

Si encadenas demasiadas operaciones o la lógica interna de cada paso es compleja, el código puede dejar de ser claro. En ese caso, conviene separar pasos intermedios.

```
const numeros = [1, 2, 3, 4, 5, 6];

const numerosPares = numeros.filter((numero) => numero % 2 === 0);
const numerosTransformados = numerosPares.map((numero) => numero * 10);

console.log(numerosTransformados);
```

14.4. Encadenamiento con objetos

```
const pokemon = [
  { nombre: "Bulbasaur", tipo: "planta", nivel: 5 },
  { nombre: "Charmander", tipo: "fuego", nivel: 8 },
  { nombre: "Squirtle", tipo: "agua", nivel: 7 }
];

const nombresFuertes = pokemon
  .filter((item) => item.nivel >= 7)
  .map((item) => item.nombre);

console.log(nombresFuertes);
```

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.filter()`
 - MDN Web Docs — `Array.prototype.map()`
-

15. Transformación de datos pensando en la interfaz

15.1. Por qué es importante

Muchas veces no transformas datos solo por operar con ellos, sino para preparar una estructura más útil para la interfaz.

15.2. Ejemplo

Supón que tienes una colección de Pokémon y quieres preparar una versión simplificada para renderizar tarjetas.

```
const pokemon = [
  {
    id: 1,
    nombre: "Bulbasaur",
    tipos: ["planta", "veneno"],
    nivel: 5
  },
  {
    id: 4,
    nombre: "Charmander",
    tipos: ["fuego"],
    nivel: 8
  }
];

const tarjetas = pokemon.map((item) => {
  return {
    titulo: item.nombre,
    subtítulo: `Nivel ${item.nivel}`,
    etiqueta: item.tipos.join(", ")
  };
});

console.log(tarjetas);
```

15.3. Qué enseña este ejemplo

- no siempre renderizas exactamente la misma estructura que recibes;
- a menudo necesitas adaptar el dato;
- separar esta transformación del renderizado mejora el diseño del código.

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.map()`
 - MDN Web Docs — Functions
-

16. Análisis técnico avanzado

16.1. Métodos declarativos frente a control manual

Muchos métodos de arrays expresan mejor la intención que un bucle manual. Por ejemplo:

- `filter()` expresa selección;
- `map()` expresa transformación;
- `find()` expresa búsqueda;
- `some()` expresa existencia;
- `every()` expresa validación total.

Esto no significa que los bucles sean incorrectos, sino que el lenguaje ofrece herramientas que comunican mejor ciertas operaciones.

16.2. La intención importa tanto como el resultado

Dos fragmentos de código pueden producir el mismo resultado, pero no describir igual de bien el problema. En proyectos reales, esa diferencia de claridad tiene un impacto directo en mantenimiento y colaboración.

16.3. Inmutabilidad no significa ineficiencia automática

Trabajar sin mutar estructuras no debe entenderse como una moda. Es una forma de reducir errores y de hacer más predecible la lógica. Aun así, debe usarse con criterio y entendiendo qué copia es superficial y qué cambios afectan a referencias internas.

16.4. Función pura no significa función trivial

Una función pura puede resolver problemas útiles y no por ello ser simplista. Lo importante es que su comportamiento sea razonable, aislado y predecible.

16.5. Testing y diseño van unidos

Cuanto mejor separada esté la lógica de transformación, más fácil será comprobarla. En muchas ocasiones, una función difícil de probar también está mal diseñada para reutilización o mantenimiento.

Referencias específicas del subtema

- MDN Web Docs — `Array.prototype.map()`
- MDN Web Docs — `Array.prototype.filter()`
- MDN Web Docs — `Array.prototype.reduce()`
- MDN Web Docs — `Set`
- MDN Web Docs — `Map`

17. Ejemplo integrador de sesión

A continuación tienes un ejemplo que reúne varios conceptos trabajados.

```
const productos = [
  { id: 1, nombre: "Teclado", precio: 30, disponible: true, categoria:
"perifericos" },
  { id: 2, nombre: "Ratón", precio: 15, disponible: false, categoria:
"perifericos" },
  { id: 3, nombre: "Monitor", precio: 200, disponible: true, categoria:
"pantallas" },
  { id: 4, nombre: "Teclado", precio: 30, disponible: true, categoria:
"perifericos" }
];

const productosDisponibles = productos.filter((producto) =>
producto.disponible);

const nombresProductos = productosDisponibles.map((producto) =>
producto.nombre);

const totalDisponibles = productosDisponibles.reduce((acumulador, producto) =>
{
  return acumulador + producto.precio;
}, 0);

const hayPantallas = productos.some((producto) => producto.categoria ===
"pantallas");

const todosDisponibles = productos.every((producto) => producto.disponible);

const nombresUnicos = [ ... new Set(productos.map((producto) =>
producto.nombre)) ];

const productosOrdenados = [ ... productos ].sort((a, b) => a.precio -
b.precio);
```



```
console.log(productosDisponibles);
console.log(nombresProductos);
console.log(totalDisponibles);
console.log(hayPantallas);
console.log(todosDisponibles);
console.log(nombresUnicos);
console.log(productosOrdenados);
```

Qué se está aplicando aquí

Elemento	Uso en el ejemplo
<code>filter()</code>	seleccionar productos disponibles
<code>map()</code>	obtener nombres
<code>reduce()</code>	calcular total acumulado
<code>some()</code>	comprobar si hay productos de una categoría
<code>every()</code>	validar una condición sobre toda la colección
<code>Set</code>	eliminar duplicados
<code>sort()</code>	ordenar sin mutar el original
Inmutabilidad	copia previa con <code>spread</code> antes de ordenar

18. Errores frecuentes al comenzar

Errores muy habituales

Estos fallos aparecen con frecuencia cuando se empieza a trabajar con métodos de arrays y transformación de datos.

18.1. Usar `forEach()` esperando un nuevo array

```
const numeros = [1, 2, 3];

const resultado = numeros.forEach((numero) => numero * 2);
```

```
console.log(resultado); // undefined
```

18.2. Usar `filter()` cuando querías un solo elemento

```
const usuarios = [  
  { id: 1, nombre: "Ana" },  
  { id: 2, nombre: "Luis" }  
];  
  
const usuario = usuarios.filter((item) => item.id === 2);  
console.log(usuario);
```

Esto devuelve un array. Si querías un único elemento, `find()` expresa mejor la intención.

18.3. Olvidar devolver valor dentro de `map()` o `reduce()`

```
const numeros = [1, 2, 3];  
  
const duplicados = numeros.map((numero) => {  
  numero * 2;  
});  
  
console.log(duplicados); // [undefined, undefined, undefined]
```

18.4. Ordenar un array sin recordar que `sort()` lo modifica

```
const numeros = [3, 1, 2];  
numeros.sort();  
  
console.log(numeros);
```

18.5. Usar `reduce()` donde una solución más simple sería más clara

Que una solución con `reduce()` sea posible no significa que sea la mejor para explicar o mantener un fragmento de código.

18.6. Mutar datos cuando querías generar una nueva versión

```
const tareas = ["estudiar", "practicar"];  
tareas.push("repasar");
```

```
console.log(tareas);
```

Si lo que querías era conservar el original y crear una nueva colección, necesitabas otra estrategia.

18.7. Escribir funciones difíciles de comprobar

Cuando una función modifica variables externas, depende de datos globales o mezcla demasiadas responsabilidades, se vuelve más difícil entenderla y validarla.

19. Síntesis de la sesión

En esta sesión has trabajado una de las bases más importantes del desarrollo moderno con JavaScript:

- recorrer colecciones con `forEach()` ;
- transformar datos con `map()` ;
- seleccionar elementos con `filter()` ;
- localizar un elemento con `find()` ;
- validar condiciones con `some()` y `every()` ;
- reducir una colección a un resultado con `reduce()` ;
- comprobar pertenencia con `includes()` ;
- ordenar resultados con `sort()` ;
- eliminar duplicados con `Set` ;
- comprender qué problema puede resolver `Map` ;
- evitar mutaciones innecesarias;
- diseñar funciones puras;
- empezar a pensar en lógica comprobable;
- adaptar datos antes de renderizarlos o procesarlos.

Esta sesión es especialmente relevante porque muchas tareas posteriores en DOM, formularios, asincronía y React dependerán de tu capacidad para transformar datos de manera clara y segura.

✓ **Al terminar esta sesión deberías ser capaz de...**

- elegir el método de array adecuado según la intención del problema;
- transformar colecciones sin alterar innecesariamente el original;

- distinguir entre selección, búsqueda, validación y acumulación;
- utilizar `Set` para eliminar duplicados en escenarios sencillos;
- comprender qué problema puede resolver `Map` ;
- aplicar una noción básica de inmutabilidad;
- reconocer una función pura y valorar por qué es más fácil de comprobar;
- empezar a razonar sobre entradas y salidas esperadas en funciones de transformación;
- preparar datos para otros procesos, como renderizado o consumo posterior.

20. Referencias documentales generales

Referencias oficiales y de consulta

- MDN Web Docs — `Array.prototype.forEach()`
- MDN Web Docs — `Array.prototype.map()`
- MDN Web Docs — `Array.prototype.filter()`
- MDN Web Docs — `Array.prototype.find()`
- MDN Web Docs — `Array.prototype.some()`
- MDN Web Docs — `Array.prototype.every()`
- MDN Web Docs — `Array.prototype.reduce()`
- MDN Web Docs — `Array.prototype.includes()`
- MDN Web Docs — `Array.prototype.sort()`
- MDN Web Docs — `Set`
- MDN Web Docs — `Map`
- MDN Web Docs — Spread syntax
- MDN Web Docs — Functions
- MDN Web Docs — `console.assert()`

Recomendación de estudio

Al repasar esta sesión, intenta describir en voz alta la intención de cada método antes de escribirlo. No empieces preguntándote “qué sintaxis uso”, sino “qué quiero hacer con los datos”: transformar, filtrar, buscar, validar, acumular o preparar una nueva estructura. Cuando la intención está clara, elegir la herramienta adecuada resulta mucho más sencillo.